

Auto Scaling TASK

Assignment Overview

This document covers building a highly available two-tier network architecture in the AWS N. Virginia (us-east-1) region: a custom VPC with one public and one private subnet, an Internet Gateway, separate public and private route tables, a NAT Gateway for private-subnet outbound access, one EC2 instance in each subnet running Apache with a sample index.html page, an Application Load Balancer distributing traffic across both instances, ALB access logs delivered to S3, VPC Flow Logs delivered to CloudWatch Logs, CloudWatch dashboards for CPU utilization and Apache service health, and an Auto Scaling policy that launches a new instance whenever CPU utilization exceeds 70 percent. Each task builds on the resources created in the prior task and is documented with its own independent steps.

The following resource reference is used throughout this document:

Task	Resource
1	VPC + public/private subnets
2	Internet Gateway
3	Public route table + private route table
4	NAT Gateway
5	Two EC2 instances (public, private)
6	Apache web server + sample index.html
7	Application Load Balancer + target group
8	S3 bucket for ALB access logs
9	VPC Flow Logs + CloudWatch Logs group
10	CloudWatch dashboards (CPU + Apache service)
11	Auto Scaling Group + CPU-based scaling policy

Task 1: Create the VPC and Public/Private Subnets

Outcome

A custom VPC named My-vpc (192.168.0.0/24) is active in us-east-1, containing one public subnet (192.168.0.0/25) in us-east-1a and one private subnet (192.168.0.128/25) in us-east-1a, both with distinct route tables to be attached in Task 2.

Objective

To establish an isolated network boundary with clearly separated public-facing and internal-only address ranges, forming the foundation the rest of the architecture is built on.

Prerequisites

- IAM permissions to create VPCs, subnets, and related networking resources.
- A chosen non-overlapping CIDR range that does not conflict with any existing VPCs to be peered later.

Step-by-Step Implementation

Step 1: Create the VPC

VPC Console -> Your VPCs -> Create VPC -> configure:

Parameter	Value
Name tag	My-vpc

Parameter	Value
IPv4 CIDR block	192.168.0.0/24
Tenancy	Default
Region	us-east-1 (N. Virginia)

Screenshot Evidence:

vpc-023e999d85505dde3 / My-vpc
Actions

Details Info

VPC ID
vpc-023e999d85505dde3

DNS resolution
Enabled

Main network ACL
acl-0939e9331ea84e0e9

IPv4 CIDR (Network border group)
-

Encryption control ID
-

State
Available

Tenancy
default

Default VPC
No

Network Address Usage metrics
Disabled

Encryption control mode
-

Block Public Access
Off

DHCP option set
dopt-0c71fc5639e429bd1

IPv4 CIDR
192.168.0.0/24

Route 53 Resolver DNS Firewall rule groups
-

DNS hostnames
Enabled

Main route table
rtb-0b1ba0dd5dc93ca43

IPv6 pool
-

Owner ID
565398309825

Resource map CIDRs Flow logs Tags Related resources Integrations

Resource map Info Show all details

Task 2: Create two subnets: one public subnet and one private subnet.

Step 1: Create the Public Subnet

VPC Console -> Subnets -> Create subnet -> configure:

Parameter	Value
Name tag	public-subnet
VPC	My-vpc
Availability Zone	us-east-1a
IPv4 CIDR block	192.168.0.0/24
Auto-assign public IPv4	Enabled

Screenshot Evidence:

subnet-0fbeaa95efc8be17c / public-subnet
Actions

Details

Subnet ID
subnet-0fbeaa95efc8be17c

IPv4 CIDR
192.168.0.0/25

Availability Zone
use1-az1 (us-east-1a)

Network ACL
-

Auto-assign customer-owned IPv4 address
No

IPv6 CIDR reservations
-

Resource name DNS AAAA record
Disabled

Subnet ARN
arn:aws:ec2:us-east-1:565398309825:subnet/subnet-0fbeaa95efc8be17c

Available IPv4 addresses
123

Network border group
us-east-1

Default subnet
No

Customer-owned IPv4 pool
-

IPv6-only
No

DNS64
Disabled

State
Available

IPv6 CIDR
-

VPC
vpc-023e999d85505dde3 | My-vpc

Auto-assign public IPv4 address
Yes

Outpost ID
-

Hostname type
IP name

Owner
565398309825

Block Public Access
Off

IPv6 CIDR association ID
-

Route table
-

Auto-assign IPv6 address
No

IPv4 CIDR reservations
-

Resource name DNS A record
Disabled

Step 2: Create the Private Subnet

VPC Console -> Subnets -> Create subnet -> configure:

Parameter	Value
Name tag	private-subnet
VPC	My-vpc
Availability Zone	us-east-1a
IPv4 CIDR block	10.0.2.0/24
Auto-assign public IPv4	Disabled

Screenshot Evidence:

The screenshot shows the AWS VPC console interface. The breadcrumb navigation at the top reads: VPC > Subnets > subnet-01d17c4865504cfc1. The main header displays 'subnet-01d17c4865504cfc1 / private-subnet' with an 'Actions' dropdown menu. On the left, the 'VPC dashboard' sidebar is visible, with 'Subnets' selected under 'Virtual private cloud'. The main content area shows the 'Details' for the subnet, organized into four columns:

- Subnet ID:** subnet-01d17c4865504cfc1
- Subnet ARN:** arn:aws:ec2:us-east-1:5653983098:subnet/subnet-01d17c4865504cfc1
- State:** Available
- Block Public Access:** Off
- IPv4 CIDR:** 192.168.0.128/25
- Available IPv4 addresses:** 123
- IPv6 CIDR:** -
- IPv6 CIDR association ID:** -
- Availability Zone:** use1-az1 (us-east-1a)
- Network border group:** us-east-1
- VPC:** vpc-023e999d85505dde3 | My-vpc
- Route table:** -
- Network ACL:** -
- Default subnet:** No
- Auto-assign public IPv4 address:** No
- Auto-assign IPv6 address:** No
- Auto-assign customer-owned IPv4 address:** No
- Customer-owned IPv4 pool:** -
- Outpost ID:** -
- IPv6 CIDR reservations:** -
- Resource name DNS AAAA record:** Disabled
- Hostname type:** IP name
- Owner:** 565398309825
- Resource name DNS A record:** Disabled

Validation Steps

- Both subnets appear under My-vpc in the Subnets list with the correct CIDR blocks.
- The public subnet shows "Auto-assign public IPv4" as Yes; the private subnet shows No.

Explanation

Splitting the /16 VPC into two /24 subnets creates separate broadcast domains that can be governed by different routing rules. The public subnet is intended to hold resources that need a direct path to the internet; the private subnet is intended to hold resources that should never be reachable directly from the internet, only able to initiate outbound connections through a NAT path configured later.

Real-Time Use Case

This public/private split is the standard baseline for any internet-facing application: a load balancer or bastion sits in the public subnet, while application servers and databases sit in the private subnet, limiting the internet-facing attack surface to only the components that require it.

Common Errors and Troubleshooting

Error	Cause	Resolution
Subnet creation fails with CIDR overlap error	Chosen subnet CIDR is not contained within, or overlaps another subnet in, the VPC CIDR	Recalculate the subnet CIDR so it falls strictly inside 10.0.0.0/16 and does not overlap the other subnet
Instances in the public subnet get no public IP	Auto-assign public IPv4 was left disabled on the subnet	Edit the subnet's IP auto-assign setting or assign an Elastic IP manually at launch

Task 3: Attach an Internet Gateway and Configure Route Tables

Outcome

An Internet Gateway named My-IGW is attached to my-vpc. A public route table sends 0.0.0.0/0 to the IGW and is associated with the public subnet; a private route table is created and associated with the private subnet, ready to receive a NAT route in Task 3.

Objective

To give the public subnet a direct path to and from the internet while keeping the private subnet's route table separate and internet-free by default.

Prerequisites

- Task 1 completed (My-vpc with public and private subnets created).
- IAM permissions to create and attach Internet Gateways and manage route tables.

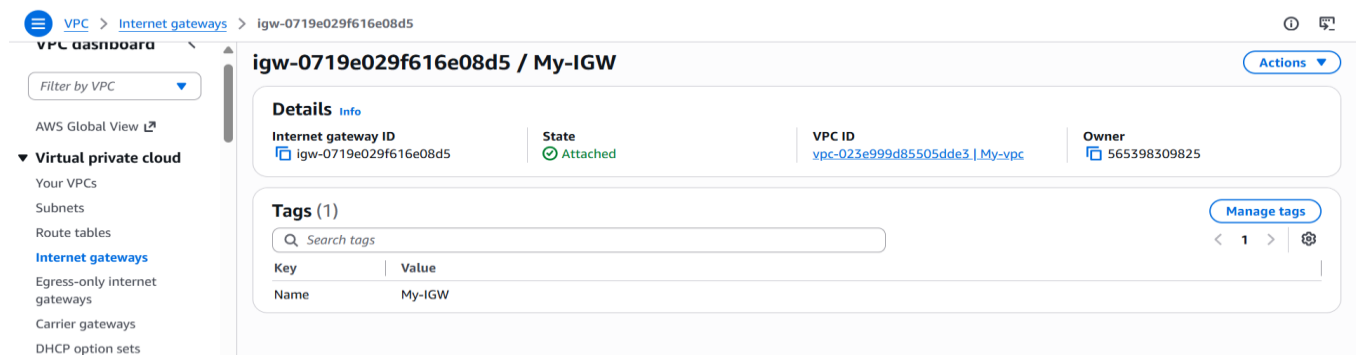
Step-by-Step Implementation

Step 1: Create and Attach the Internet Gateway

VPC Console -> Internet Gateways -> Create internet gateway -> name it My-IGW -> Create. Select it -> Actions -> Attach to VPC -> choose My-vpc.

Parameter	Value
Name tag	My-IGW
Attached VPC	My-vpc

Screenshot Evidence:



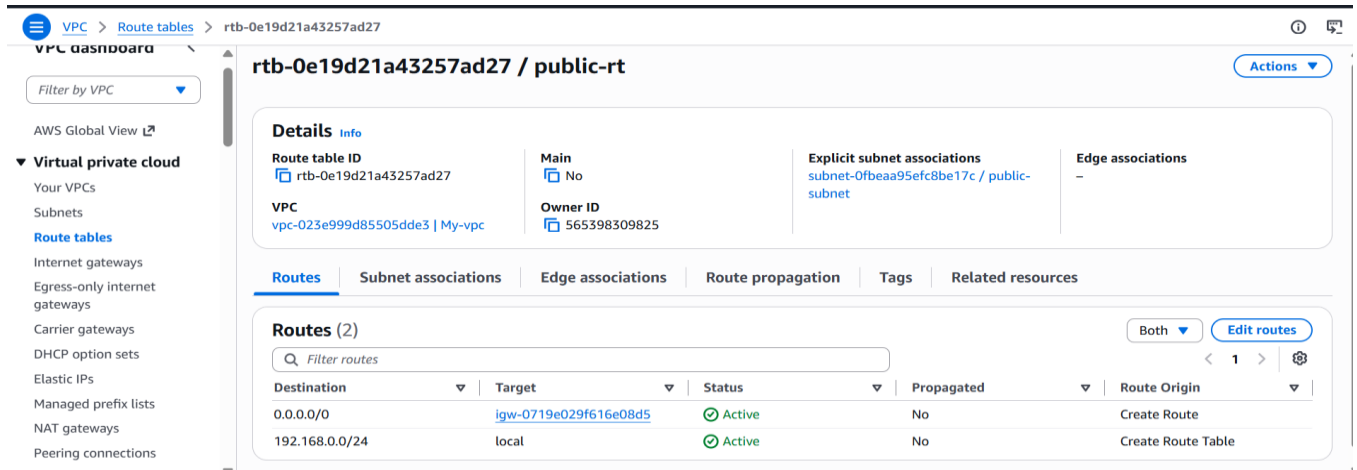
Task 4 : Create one public route table (RT) and one private route table

Step 1: Create the Public Route Table

VPC Console -> Route Tables -> Create route table -> configure, then add the internet route and associate the public subnet:

Parameter	Value
Name tag	public-rt
VPC	My-vpc
Route	0.0.0.0/0 -> My-IGW
Subnet association	public-subnet

Screenshot Evidence:

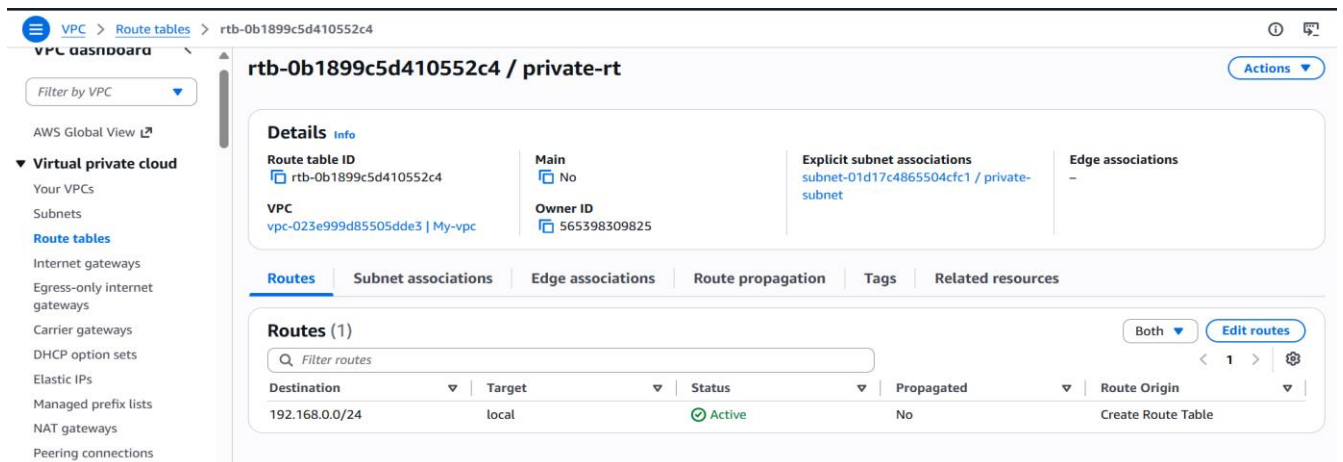


Step 2: Create the Private Route Table

VPC Console -> Route Tables -> Create route table -> configure and associate the private subnet (no internet route yet):

Parameter	Value
Name tag	private-rt
VPC	My-vpc
Subnet association	private-subnet

Screenshot Evidence:



Validation Steps

- The public route table's Routes tab shows a 0.0.0.0/0 entry targeting lab-igw with state Active.
- The Subnet Associations tab confirms lab-public-subnet is tied to lab-public-rt and lab-private-subnet is tied to lab-private-rt.

Explanation

The Internet Gateway is a horizontally scaled, VPC-attached component that performs network address translation for instances with public IPs. A route table only grants internet access if it contains an explicit route to the IGW; because the private route table intentionally omits that route, resources in the private subnet remain unreachable from, and unable to reach, the internet directly.

Real-Time Use Case

Separating route tables per tier is what prevents a misconfigured private-subnet instance from being exposed to the internet by accident; any future internet-bound route added to the private table (such as the NAT route in Task 3) is a deliberate, auditable change rather than an inherited default.

Common Errors and Troubleshooting

Error	Cause	Resolution
Public subnet instances still cannot reach the internet	Route table not associated with the subnet, or the 0.0.0.0/0 route is missing	Confirm the subnet association tab and re-add the IGW route if absent
Internet Gateway attach fails	IGW is already attached to a different VPC	Detach it from the other VPC first, or create a new IGW dedicated to lab-vpc

Task 5: Deploy a NAT Gateway for Private Subnet Internet Access

Outcome

A NAT Gateway named lab-nat-gw is deployed in the public subnet with an Elastic IP, and the private route table is updated so all outbound internet-bound traffic from the private subnet (0.0.0.0/0) is routed through it.

Objective

To let instances in the private subnet initiate outbound connections (for example, OS package updates) without ever accepting inbound connections from the internet.

Prerequisites

- Task 2 completed (IGW attached, public and private route tables in place).
- An Elastic IP available or able to be allocated for the NAT Gateway.

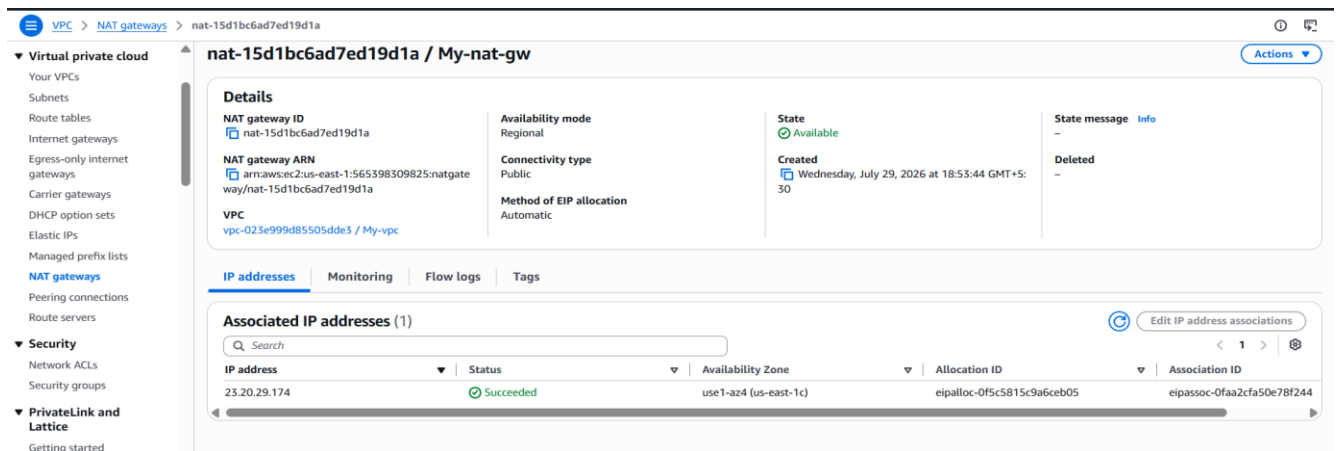
Step-by-Step Implementation

Step 1: Allocate an Elastic IP and Create the NAT Gateway

VPC Console -> NAT Gateways -> Create NAT gateway -> configure:

Parameter	Value
Name tag	My-nat-gw
Subnet	public-subnet
Connectivity type	Public
Elastic IP allocation	Allocate new EIP

Screenshot Evidence:

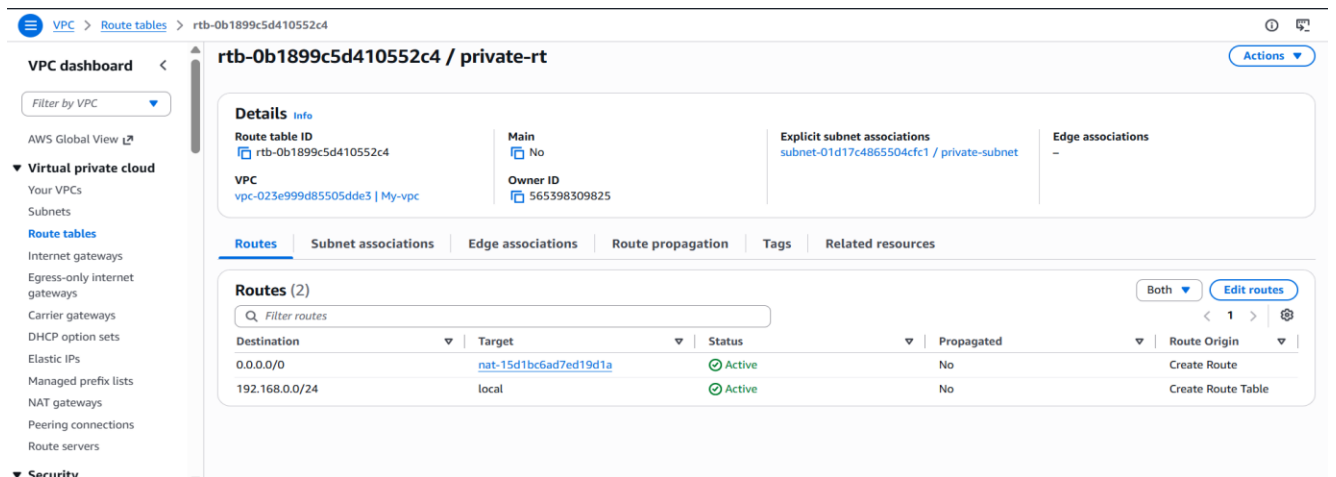


Step 2: Route the Private Subnet Through the NAT Gateway

VPC Console -> Route Tables -> select private-rt -> Routes -> Edit routes -> Add route:

Parameter	Value
Destination	0.0.0.0/0
Target	My-nat-gw

Screenshot Evidence:



Validation Steps

- The NAT Gateway state shows Available in the console.
- lab-private-rt's Routes tab shows 0.0.0.0/0 targeting the NAT Gateway ID.
- An instance in the private subnet can reach the internet outbound (for example, a package manager update succeeds) while remaining unreachable by direct inbound connection from the internet.

Explanation

A NAT Gateway sits in a public subnet, holds its own Elastic IP, and translates outbound traffic from private-subnet instances so replies can find their way back, while never allowing an internet host to initiate a connection inward. This preserves the private subnet's isolation while still allowing it to fetch updates, call external APIs, or reach other AWS services.

Real-Time Use Case

Application and database tiers that must never be directly internet-reachable, but still need to download patches or call third-party APIs, rely on exactly this pattern; it is the standard way to give a private subnet controlled, one-directional internet access.

Common Errors and Troubleshooting

Error	Cause	Resolution
NAT Gateway stuck in Pending	Elastic IP allocation still in progress or subnet has no available IP	Wait for provisioning to complete; confirm the public subnet has a free address
Private instance still has no outbound internet access	Private route table not updated, or NAT Gateway placed in the private subnet instead of the public one	Verify the 0.0.0.0/0 route targets the NAT Gateway and that the NAT Gateway itself lives in the public subnet
Unexpected data-transfer charges	NAT Gateway processes and charges for all data through it	Route only the traffic that needs it, and monitor NAT Gateway CloudWatch metrics for volume

Task 6: Launch EC2 Instances in the Public and Private Subnets

Outcome

Two EC2 instances are running: lab-public-ec2 in lab-public-subnet with a public IP and SSH/HTTP access, and lab-private-ec2 in lab-private-subnet with no public IP, reachable only from inside the VPC.

Objective

To provision the compute resources that will each run Apache and later sit behind the Application Load Balancer, with the private instance demonstrating that it is reachable purely through internal networking and the NAT path.

Prerequisites

- Tasks 1-3 completed (subnets, route tables, and NAT Gateway in place).
- A key pair available for SSH access.
- A security group allowing HTTP (80) and SSH (22, scoped appropriately).

Step-by-Step Implementation

Step 1: Create a Security Group for the Instances

EC2 Console -> Security Groups -> Create security group -> configure inbound rules:

Parameter	Value
Name	publicEC2
VPC	My-vpc
Inbound rule 1	HTTP (80) from 0.0.0.0/0
Inbound rule 2	SSH (22) from admin IP/CIDR only

Screenshot Evidence:

Step 2: Launch the Public EC2 Instance

EC2 Console -> Launch instance -> configure:

Parameter	Value
Name	publicEC2
AMI	Amazon Linux 2025
Instance type	t3.micro
Subnet	public-subnet
Auto-assign public IP	Enable
Security group	lanuch-10

Screenshot Evidence:

▼ Network settings

Info

VPC - required

Info

vpc-023e999d85505dde3 (My-vpc)

192.168.0.0/24

▼

↻

Subnet

Info

subnet-0fbeaa95efc8be17c

public-subnet

▼

↻ Create new subnet

Auto-assign public IP

Info

Enable

▼

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

Security group name - required

launch-10

EC2 > Instances > i-068761212753e4bc8

EC2

Dashboard

AWS Global View

Events

▼ Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

▼ Images

AMIs

Instance summary for i-068761212753e4bc8 (publicEC2)

Info

↻

Connect

Instance state ▼

Actions ▼

Updated less than a minute ago

Instance ID

i-068761212753e4bc8

Public IPv4 address

52.200.210.249 | open address

Private IPv4 addresses

192.168.0.31

IPv6 address

-

Instance state

Running

Public DNS

ec2-52-200-210-249.compute-1.amazonaws.com | open address

Hostname type

IP name: ip-192-168-0-31.ec2.internal

Private IP DNS name (IPv4 only)

ip-192-168-0-31.ec2.internal

Answer private resource DNS name

-

Instance type

t3.micro

Auto-assigned IP address

52.200.210.249 [Public IP]

VPC ID

vpc-023e999d85505dde3 (My-vpc)

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Step 3: Launch the Private EC2 Instance

EC2 Console -> Launch instance -> configure:

Parameter	Value
Name	privateEC2
AMI	Amazon Linux 2025
Instance type	t3.micro
Subnet	lab-private-subnet
Auto-assign public IP	Disable
Security group	lanuch10

Screenshot Evidence:

EC2

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

EC2 > Instances > i-052111559602f4fd3

Instance summary for i-052111559602f4fd3 (privateEC2)

Updated less than a minute ago

Instance ID

i-052111559602f4fd3

IPv6 address

-

Hostname type

IP name: ip-192-168-0-220.ec2.internal

Answer private resource DNS name

-

Auto-assigned IP address

-

IAM role

-

Public IPv4 address

-

Instance state

Running

Private IP DNS name (IPv4 only)

ip-192-168-0-220.ec2.internal

Instance type

t3.micro

VPC ID

vpc-023e999d8550dde3 (My-vpc)

Subnet ID

subnet-01d17c4865504fc1 (private-subnet)

Private IPv4 addresses

192.168.0.220

Public DNS

-

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations.

Learn more

Auto Scaling Group name

-

Connect

Instance state

Actions

Placing one instance in each subnet exercises both halves of the routing configuration built in Tasks 1-3: the public instance proves the IGW path works, and the private instance proves the NAT path works for outbound traffic while inbound access from the internet stays blocked by both the subnet's routing and the security group.

Real-Time Use Case

This mirrors a common two-tier deployment where a public instance might serve as a bastion or edge component and the private instance runs internal application logic, both of which will shortly be fronted by the same load balancer to demonstrate ALB-to-private-target routing.

Common Errors and Troubleshooting

Error	Cause	Resolution
Cannot SSH into public instance	Security group missing inbound rule 22, or wrong key pair used	Add the SSH inbound rule scoped to the admin IP and confirm the correct .pem file
Private instance unreachable from Session Manager	IAM instance profile missing AmazonSSMMangedInstanceCore, or no route to the Session Manager endpoint	Attach the SSM managed policy to the instance role; confirm NAT Gateway route is active

Task 7: Deploy Apache Web Server on Both EC2 Instances

Outcome

Apache (httpd) is installed, enabled, and running on both lab-public-ec2 and lab-private-ec2, each serving a distinct sample index.html page identifying which instance answered the request.

Objective

To give both instances a working HTTP service that the Application Load Balancer can health-check and route traffic to in Task 6.

Prerequisites

- Task 4 completed (both EC2 instances running).
- SSH or Session Manager access to each instance.

Step-by-Step Implementation

Step 1: Install and Start Apache on the Public Instance

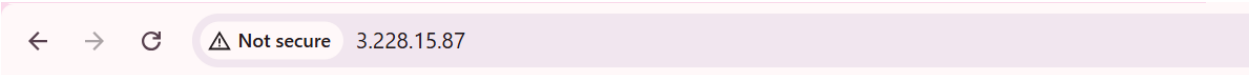
```
sudo yum update -y
sudo yum install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
echo '<html><body><h1>Hello from Public EC2 Instance</h1></body></html>' | sudo tee /var/www/html/index.html
```

Screenshot Evidence:

```
[ec2-user@ip-192-168-0-65 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-192-168-0-65 ~]$ sudo yum install httpd -y
Last metadata expiration check: 0:01:14 ago on Wed Jul 29 16:30:50 2026.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
httpd	x86_64	2.4.68-1.amzn2023.0.1	amazonlinux	46 k
Installing dependencies:				
apr	x86_64	1.7.5-1.amzn2023.0.4	amazonlinux	129 k
apr-util	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	97 k
apr-util-ldap	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	13 k
generic-logos-httpd	noarch	18.0.0-12.amzn2023.0.3	amazonlinux	19 k
httpd-core	x86_64	2.4.68-1.amzn2023.0.1	amazonlinux	1.4 M
httpd-filesystem	noarch	2.4.68-1.amzn2023.0.1	amazonlinux	12 k
httpd-tools	x86_64	2.4.68-1.amzn2023.0.1	amazonlinux	80 k
libbrotli	x86_64	1.0.9-4.amzn2023.0.2	amazonlinux	315 k
mailcap	noarch	2.1.49-3.amzn2023.0.3	amazonlinux	33 k
Installing weak dependencies:				
apr-util-openssl	x86_64	1.6.3-1.amzn2023.0.2	amazonlinux	15 k
mod_http2	x86_64	2.0.42-1.amzn2023.0.1	amazonlinux	167 k

```
Complete!
[ec2-user@ip-192-168-0-65 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-192-168-0-65 ~]$ sudo systemctl start httpd
[ec2-user@ip-192-168-0-65 ~]$ sudo tee /var/www/html/index.html > /dev/null <<EOF
<html>
<head>
<title>Apache</title>
</head>
<body>
<h1>Hello from Public EC2 Instance</h1>
</body>
</html>
EOF
[ec2-user@ip-192-168-0-65 ~]$ sudo systemctl restart httpd
```



Hello from Public EC2 Instance

Step 2: Install and Start Apache on the Private Instance

```
sudo yum update -y
sudo yum install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
echo '<html><body><h1>Hello from Private EC2 Instance</h1></body></html>' | sudo tee /var/www/html/index.html
```

Screenshot Evidence:

```

      #
    ~\##### Amazon Linux 2023
    ~~\#####
    ~~~\####|
    ~~~~~\#/ https://aws.amazon.com/linux/amazon-linux-2023
    ~~~~~V~' '->
    ~~~~~
    ~~~~~.
    ~~~~~/_m/' -/_
Last login: Wed Jul 29 16:29:20 2026 from 18.206.107.29
[ec2-user@ip-192-168-0-65 ~]$ ll
total 4
-r----- . 1 ec2-user ec2-user 1678 Jul 29 16:43 multitool-key.pem

```

```
~\ #### Amazon Linux 2023
~\ \#####\
~~\ \####|
~~\ \|/ https://aws.amazon.com/linux/amazon-linux-2023
~~~~ V ~' ->
~~~~ 
~~~~ _ . /_
~/m/' -
```

Last login: Wed Jul 29 16:50:14 2026 from 192.168.0.65
[ec2-user@ip-192-168-0-192 ~]\$ sudo systemctl status httpd

```
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-07-29 16:52:01 UTC; 4min 13s ago
     Docs: man:httpd.service(8).
 Main PID: 26111 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
      Tasks: 177 (limit: 1059)
     Memory: 13.9M
        CPU: 395ms
CGroup: /system.slice/httpd.service
├─26111 /usr/sbin/httpd -DFOREGROUND
├─26112 /usr/sbin/httpd -DFOREGROUND
├─26113 /usr/sbin/httpd -DFOREGROUND
├─26114 /usr/sbin/httpd -DFOREGROUND
└─26169 /usr/sbin/httpd -DFOREGROUND
```

Jul 29 16:52:01 ip-192-168-0-192.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jul 29 16:52:01 ip-192-168-0-192.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jul 29 16:52:01 ip-192-168-0-192.ec2.internal httpd[26111]: Server configured, listening on: port 80

```
[ec2-user@ip-192-168-0-192 ~]$ curl curl http://192.168.0.192
curl: (6) Could not resolve host: curl
<html>
<head>
<title>Apache</title>
</head>
<body>
<h1>Hello from Private EC2 Instance</h1>
</body>
</html>
[ec2-user@ip-192-168-0-192 ~]$
```

Validation Steps

- `curl http://localhost/` on each instance returns its respective sample HTML.
- Browsing to the public instance's public IP in a browser displays the Public EC2 Instance page.
- `systemctl status httpd` shows active (running) on both instances.

Explanation

Apache is installed identically on both instances via the OS package manager, then enabled so it restarts automatically after a reboot. Serving a distinct index.html per instance makes it possible to visually confirm, once the load balancer is in front of both, that requests are actually being distributed across both targets rather than always hitting one.

Real-Time Use Case

This is the simplest possible representative workload for validating a load-balanced, multi-AZ or multi-subnet web tier before deploying a real application; the same steps generalize directly to installing and starting any other web server or application runtime.

Common Errors and Troubleshooting

Error	Cause	Resolution
Apache installed but page unreachable externally	Security group does not allow inbound HTTP 80	Add an inbound rule for TCP 80 from 0.0.0.0/0 on lab-web-sg
systemctl enable httpd has no effect after reboot	Service was started but not enabled	Re-run sudo systemctl enable httpd and verify with systemctl is-enabled httpd

Task 8: Deploy an Application Load Balancer in Front of Both Instances

Outcome

An internet-facing Application Load Balancer named lab-alb is created in the public subnet, forwarding HTTP traffic to a target group named lab-tg that contains both lab-public-ec2 and lab-private-ec2 as registered, healthy targets.

Objective

To distribute incoming HTTP traffic across both instances and provide a single, stable DNS entry point, decoupling clients from any individual instance's IP address.

Prerequisites

- Task 5 completed (Apache running and serving index.html on both instances).
- At least two subnets in different Availability Zones are recommended for production; this lab uses the existing public subnet for the ALB nodes and registers both instances as targets by IP or instance ID.

Step-by-Step Implementation

Step 1: Create the Target Group

EC2 Console -> Target Groups -> Create target group -> configure:

Parameter	Value
Target type	Instances
Name	apache-target-group
Protocol : Port	HTTP : 80
VPC	lab-vpc
Health check path	/
Registered targets	public-EC2, private-EC2

Screenshot Evidence:

EC2 > Target groups > apache-target-group

apache-target-group

Details

arn:aws:elasticloadbalancing:us-east-1:565398309825:targetgroup/apache-target-group/f09919cfd8c46a8

Target type: Instance

Protocol : Port: HTTP: 80

Protocol version: HTTP1

VPC: vpc-023e999d85505dde3

IP address type: IPv4

Load balancer: None associated

2 Total targets

0 Healthy

0 Unhealthy

2 Unused

0 Initial

0 Draining

0 Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (2) [Info](#) [Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Administrati...	Override d
☐	i-0c2a9697a40ab355e	privateEC2	80	us-east-1a (us...	Unused	Target group is not co...	-	-
☐	i-06dce8abcc4d8eaaa	publicEc2	80	us-east-1a (us...	Unused	Target group is not co...	-	-

Step 2: Create the Application Load Balancer

EC2 Console -> Load Balancers -> Create load balancer -> Application Load Balancer -> configure:

Parameter	Value
Name	My-alb
Scheme	Internet-facing
VPC	My-vpc
Subnets	public-subnet (+ a second AZ subnet if available)
Security group	web-sg
Listener	HTTP : 80 -> forward to apcahe-target-group

Screenshot Evidence:

request, it evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.

How Application Load Balancers work

Basic configuration

Load balancer name

Name must be unique within your AWS account and can't be changed after the load balancer is created.

My-alb

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme [Info](#)

Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves Internet-facing traffic.
- Has public IP addresses.
- DNS name resolves to public IPs.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name resolves to private IPs.
- Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type [Info](#)

Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**

Includes only IPv4 addresses.

☐ **Dualstack**

Includes IPv4 and IPv6 addresses.

Network mapping Info

The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC Info

The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#).

vpc-023e999d85505dde3 (My-vpc)

192.168.0.0/24

Create VPC

IP pools Info

You can optionally choose to configure an IPAM pool as the preferred source for your load balancers IP addresses. Create or view [Pools](#) in the [Amazon VPC IP Address Manager console](#).

☐ Use IPAM pool for public IPv4 addresses

The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.

Availability Zones and subnets Info

Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ us-east-1a (use1-az1)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-0fbeaa95efc8be17c (public-subnet)

192.168.0.0/25

Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (✓)

My-ALB ⚠ Reachability may be impacted

Actions

▼ Details

Load balancer type Application	Status Active	VPC vpc-023e999d85505dde3	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z35XDOTRQ7X7K	Availability Zones subnet-0fbeaa95efc8be17c us-east-1a (use1-az1) subnet-078fe00b0bd33bcea us-east-1b (use1-az2)	Date created July 29, 2026, 23:15 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:565398309825:loadbalancer/app/My-ALB/7491e8b406ea245f		DNS name Info My-ALB-1793123762.us-east-1.elb.amazonaws.com (A Record)	

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Listeners and rules (1) Info

Manage rules

Manage listener

Add listener

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

← → ↺

⚠ Not secure

my-clb-989991990.us-east-1.elb.amazonaws.com

Hello from Public EC2 Instance

Validation Steps

- Both targets show health status Healthy in the lab-tg Targets tab.
- Refreshing the ALB's DNS name in a browser alternates between the Public and Private instance's sample page across repeated requests.
- The load balancer state shows Active.

Explanation

The ALB terminates client connections and forwards them to whichever registered, healthy target the routing algorithm selects, health-checking each target on the configured path so an unhealthy instance is automatically taken out of rotation. Because the private instance's target registration uses its private IP within the VPC, the ALB can reach it directly without the private instance needing any public exposure.

Real-Time Use Case

This is the standard pattern for exposing a multi-instance application tier under one endpoint: clients only ever see the ALB's DNS name, instances can be added, removed, or replaced behind it without any client-facing change, and unhealthy instances are automatically routed around.

Common Errors and Troubleshooting

Error	Cause	Resolution
Target shows Unhealthy	Security group blocks the ALB's traffic, or health check path returns non-200	Allow inbound HTTP from the ALB security group on lab-web-sg and confirm the health check path exists
ALB provisioning fails	Fewer than two subnets in two distinct Availability Zones supplied	Add a subnet from a second Availability Zone to the ALB's subnet mapping

Task 9: Store Application Load Balancer Access Logs in S3

Outcome

Access logging is enabled on My-alb, delivering gzip-compressed log files for every processed request into a dedicated, encrypted S3 bucket under a defined prefix.

Objective

To retain a durable, queryable record of every request the ALB processes for troubleshooting, security review, and traffic analysis.

Prerequisites

- Task 6 completed (My-ALB active with healthy targets).
- IAM permissions to create an S3 bucket and modify its bucket policy.

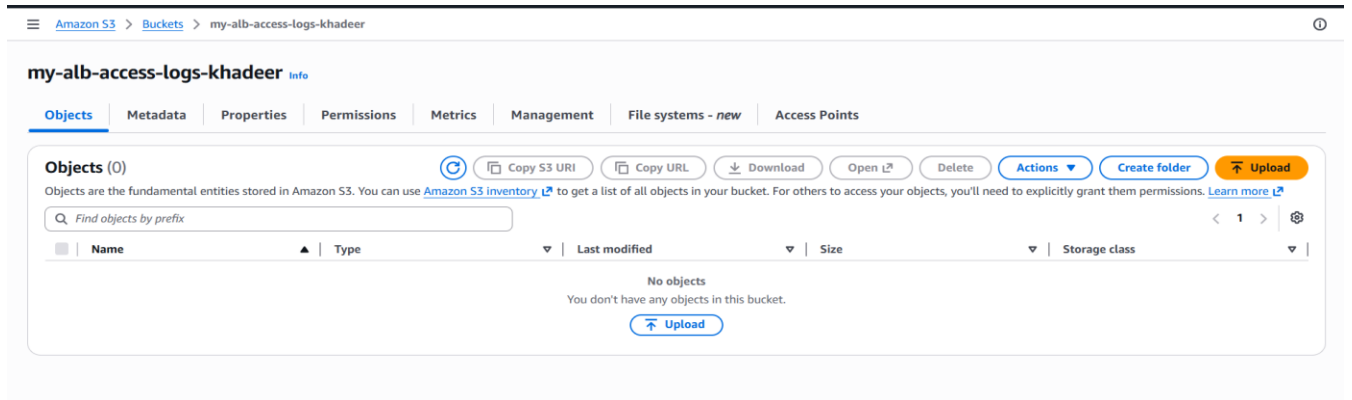
Step-by-Step Implementation

Step 1: Create the S3 Bucket for ALB Logs

S3 Console -> Create bucket -> configure:

Parameter	Value
Bucket name	my-alb-access-logs
Region	us-east-1
Block Public Access	Enabled (all four settings)
Default encryption	SSE-S3

Screenshot Evidence:



Step 2: Enable Access Logs on the ALB

EC2 Console -> Load Balancers -> lab-alb -> Attributes -> Edit -> configure:

Parameter	Value
Monitoring: Access logs	Enabled
S3 bucket	my-alb-access-logs--khadeer
Prefix	alb-logs

Screenshot Evidence:

Logging

All legacy logs are stored in S3. If no prefix is specified, logs are stored in the bucket root.

☒ Access logs - *legacy*

Access logs deliver detailed logs of all requests made to your load balancer. Additional charges apply. [Learn more](#)

S3 URI

Q s3://my-alb-access-logs-khadeer



View

Browse S3

Format: s3://bucket/prefix

☐ Connection logs - *legacy*

Connection logs deliver detailed logs of all connections made to your load balancer. Additional charges apply. [Learn more](#)

☐ Health check logs - *legacy*

Health check logs deliver detailed logs of health checks for all targets in your load balancer's target groups. Additional charges apply. [Learn more](#)

Logging options

Log delivery to CloudWatch, Kinesis Firehose, and S3 is configurable through an integration with Logging - by Amazon CloudWatch. Visit **Logging** in the [Integrations tab](#).

Cancel

Save changes

Amazon S3 > Buckets > my-alb-access-logs-khadeer > AWSLogs/ > 565398309825/

565398309825/

Copy S3 URI

Objects

Properties

Objects (1)



Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Show versions

1



Name



Type



Last modified



Size



Storage class



ELBAccessLogTestFile

-

July 29, 2026, 23:53:56 (UTC+05:30)

81.0 B

Standard

Explanation

When access logging is enabled, AWS automatically attaches the bucket policy the ALB's service account needs to write objects, then periodically batches and delivers logs as compressed files containing per-request details such as client IP, target, response time, and response code.

Real-Time Use Case

Operations teams query these logs to investigate error-rate spikes, unusual client behavior, or slow response times after the fact, and security teams use them to reconstruct the request history around an incident, all without needing to instrument the application itself.

Common Errors and Troubleshooting

Error	Cause	Resolution
No log files appear in the bucket	Bucket policy missing the ALB service account's PutObject permission	Re-run the enable-logging step through the console so it re-attaches the required bucket policy automatically
Access denied enabling logging	Bucket has Block Public Access misconfigured or is in a different region than the ALB	Create the bucket in the same region as the ALB and keep default Block Public Access settings

Task 10: Store VPC Flow Logs in a CloudWatch Log Group

Outcome

A VPC Flow Log is enabled on lab-vpc, streaming all accepted and rejected traffic metadata into a CloudWatch Logs log group named /vpc/lab-vpc/flow-logs.

Objective

To capture network-level visibility into every IP flow entering and leaving the VPC, independent of and complementary to the ALB's request-level logs.

Prerequisites

- Task 1 completed (my-vpc created).
- An IAM role that the Flow Logs service can assume to write into CloudWatch Logs.

Step-by-Step Implementation

Step 1: Create the CloudWatch Logs Log Group

CloudWatch Console -> Log groups -> Create log group -> configure:

Parameter	Value
Log group name	My-ALB-loggroup
Retention	14 days (or as required)

Screenshot Evidence:

The screenshot displays the AWS CloudWatch console for a log group named 'My-ALB-Loggroup'. At the top, there are buttons for 'Actions', 'View in Logs Insights', 'Start tailing', and 'Search log group'. Below these, the 'Log group details' section is expanded, showing various configuration options:

- Log class:** Standard (with an 'Info' link).
- ARN:** arn:aws:logs:us-east-1:565398309825:log-group:My-ALB-Loggroup:*
- Creation time:** Now
- Retention:** 2 weeks
- Stored bytes:** -
- Metric filters:** 0
- Subscription filters:** 0
- Contributor Insights rules:** -
- KMS key ID:** -
- Deletion protection:** Off (with a toggle icon)
- Data protection:** -
- Sensitive data count:** -
- Custom field indexes:** Configure (link)
- Transformer:** Configure (link)
- Anomaly detection:** Configure (link)
- Bearer token authentication:** Off (with a toggle icon)
- Syslog ingestion:** Info (link) and Configure (link)

At the bottom, a horizontal navigation bar includes tabs for 'Log streams', 'Tags', 'Data protection', 'Anomaly detection', 'Metric filters', 'Subscription filters', 'Contributor Insights', and 'Field indexes'. The 'Log streams' tab is currently selected.

Step 2: Create the VPC Flow Log

VPC Console -> select My-vpc -> Flow Logs -> Create flow log -> configure:

Parameter	Value
Filter	All (accept + reject)
Maximum aggregation interval	1 minute
Destination	Send to CloudWatch Logs
Destination log group	My-ALB-loggroup
IAM role	flow-logs-to-cloudwatch-role (auto-created via wizard)

Screenshot Evidence:

VPC

Your VPCs

Create flow logs

Create flow log

Info

Flow logs can capture IP traffic flow information for the network interfaces associated with your resources. You can create multiple flow logs to send traffic to different destinations.

Selected resources

Info

Flow logs will only be created for resources in an available state.

Name	Resource ID	State
My-vpc	vpc-023e999d85505dde3	Available

Flow log settings

Name - optional

my-flow-log-01

Filter

The type of traffic to capture (accepted traffic only, rejected traffic only, or all traffic).

Accept

Reject

All

Maximum aggregation interval

Info

The maximum interval of time during which a flow of packets is captured and aggregated into a flow log record.

10 minutes

1 minute

Destination

Destination

The destination to which to publish the flow log data.

Send to CloudWatch Logs

Send to an Amazon S3 bucket

Send to Amazon Data Firehose in the same account

Send to Amazon Data Firehose in a different account

Destination log group

Info

The name of an existing log group or the name of a new log group that will be created when you create this flow log. A new log stream is created for each monitored network interface.

My-ALB-Loggroup

View this log group in the CloudWatch console

Service access

VPC flow logs require permissions to create log groups and publish events in CloudWatch.

Use an existing service role

Create and use a new service role

Service role name

Info

VPCFlowLogs-Cloudwatch-1785350431204

Log record format

Specify the fields to include in the flow log record.

AWS default format

Custom format

Additional metadata

Include additional metadata to AWS default log record format.

fl-00c341ef61b93a6da

Actions

Details

Flow Log ID

fl-00c341ef61b93a6da

Name

-

State

Active

Creation Time

Thursday, July 30, 2026 at 00:12:41 GMT+5:30

Destination Type

cloud-watch-logs

Destination Name

My-ALB-Loggroup

IAM Role

arn:aws:iam::565398309825:role/service-role/VPCFlowLogs-Cloudwatch-1785350431204

Cross Account IAM Role

-

Traffic Type

All

Max Aggregation Interval

1 minute

Log Format

Default

Tag field specifications

-

File Format

-

Hive Compatible Partitions

-

Partition Logs

-

Tags

Integrations

Tags

Manage tags

Search tags

Key

Value

Validation Steps

- The Flow Logs tab on My-vpc shows status Active.
- New log streams, one per elastic network interface, appear inside /vpc/My-vpc/flow-log within a few minutes.
- Log entries show ACCEPT and REJECT actions matching real traffic to and from the instances.

Explanation

Flow Logs capture IP traffic metadata (source/destination IP and port, protocol, byte counts, and accept/reject action) at the elastic network interface level, independent of any application or load balancer. Sending them to CloudWatch Logs, rather than only to S3, makes them immediately searchable with Logs Insights and usable as an input to metric filters and alarms.

Real-Time Use Case

Security teams use Flow Logs to confirm whether a security group change actually blocked the traffic it was meant to, and network teams use them to diagnose connectivity issues (for example, confirming that private-subnet traffic is correctly egressing through the NAT Gateway) without needing access to the instances themselves.

Common Errors and Troubleshooting

Error	Cause	Resolution
Flow log creation fails with IAM error	No IAM role with permission to publish to CloudWatch Logs	Let the console auto-create the flow-logs-to-cloudwatch-role, or attach the required log-publishing policy manually
Log group receives no data	Flow log filter or destination misconfigured	Confirm the flow log status is Active and the destination log group name matches exactly

Task 11: Build CloudWatch Dashboards for CPU Utilization and Apache Service Health

Outcome

A CloudWatch dashboard named lab-monitoring-dashboard displays CPU utilization for both EC2 instances and a widget showing whether the Apache (httpd) process is running on each, using the CloudWatch Agent's procstat plugin for the service-health metric.

Objective

To give a single at-a-glance view of both infrastructure-level health (CPU) and application-level health (whether Apache is actually running), rather than relying on CPU alone, which cannot detect a crashed web server on an otherwise healthy instance.

Prerequisites

- Task 5 completed (Apache running on both instances).
- An IAM role with CloudWatchAgentServerPolicy attached to both instances.
- The CloudWatch Agent installed and configured with a procstat block targeting the httpd process.

Step-by-Step Implementation

Step 1: Attach the CloudWatch Agent IAM Role and Install the Agent

Attach CloudWatchAgentServerPolicy to each instance's IAM role first (EC2 Console -> instance -> Actions -> Security -> Modify IAM role). During the config wizard, add a procstat entry monitoring the pattern httpd so the running process count is published as a custom metric.

```
sudo yum install -y amazon-cloudwatch-agent
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

Screenshot Evidence:

```
[ec2-user@ip-192-168-0-234 ~]$ sudo yum update -y
Last metadata expiration check: 1:54:47 ago on Thu Jul 30 09:43:15 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-192-168-0-234 ~]$ sudo yum install -y amazon-cloudwatch-agent
Last metadata expiration check: 1:54:59 ago on Thu Jul 30 09:43:15 2026.
Dependencies resolved.

Package                               Architecture      Version           Repository        Size
Installing:
amazon-cloudwatch-agent              x86_64            1.300067.1-1.amzn2023      amazonlinux        68 M

Transaction Summary
-----
Install 1 Package

Total download size: 68 M
Installed size: 231 M
Downloading Packages:
amazon-cloudwatch-agent-1.300067.1-1.amzn2023.x86_64.rpm                                68 MB/s | 68 MB | 00:01
-----
Total                                                                                      65 MB/s | 68 MB | 00:01
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      : 1/1
```

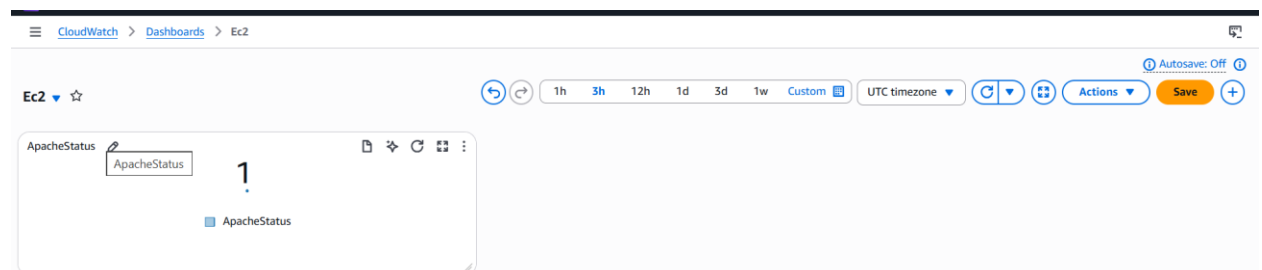
```
[ec2-user@ip-192-168-0-234 ~]$ sudo systemctl status amazon-cloudwatch-agent
● amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
   Loaded: loaded (/etc/systemd/system/amazon-cloudwatch-agent.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-07-30 11:45:06 UTC; 12s ago
     Main PID: 34261 (amazon-cloudwat)
       Tasks: 7 (limit: 1059)
      Memory: 47.0M
         CPU: 316ms
    CGroup: /system.slice/amazon-cloudwatch-agent.service
            └─34261 /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml -envconfig /opt/aws

Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/ama
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json does not exist or cannot
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/ama
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 I! Valid json input schema.
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: I! Trying to detect region from ec2
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 D! ec2tagger processor required because metrics with diskio or net are set
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 D! ec2tagger processor required because metrics with diskio or net are set
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34271]: 2026/07/30 11:45:06 Configuration validation first phase succeeded
Jul 30 11:45:06 ip-192-168-0-234.ec2.internal start-amazon-cloudwatch-agent[34261]: I! Detecting run_as_user...
```

Step 2: Create the Dashboard

CloudWatch Console -> Dashboards -> Create dashboard -> name Apache -> add widgets:

Screenshot Evidence:



Validation Steps

- The CPU widget updates roughly every 60 seconds and reflects load changes on either instance.

- The Apache widget shows a process count of 1 while httpd is running and drops to 0 within about a minute of stopping the service (for example, via `sudo systemctl stop httpd`).

Explanation

Standard EC2 metrics report CPUUtilization automatically with no agent required, but they say nothing about what is happening inside the guest OS. The CloudWatch Agent's procstat plugin counts how many running processes match a given name pattern and reports that count as a custom metric, so the dashboard can distinguish between a healthy instance running Apache and a healthy instance whose Apache process has crashed or been stopped.

Real-Time Use Case

Operations teams use exactly this combination to catch the common failure mode where the underlying EC2 instance passes every infrastructure health check while the web server process itself has died, a condition basic CPU or status-check monitoring alone would never surface.

Common Errors and Troubleshooting

Error	Cause	Resolution
procstat metric never appears	Process name pattern in the agent config does not match the actual running process	Verify with <code>ps aux grep httpd</code> and correct the pattern in the agent's JSON config, then restart the agent
CPU widget shows no data for one instance	Dashboard widget references the wrong InstanceId	Edit the widget and reselect the correct instance dimension

Task 12: Configure Auto Scaling to Launch a New Instance Above 70% CPU

Outcome

The public EC2 instance is converted into an Auto Scaling Group (lab-asg, min 1 / desired 1 / max 3) behind a launch template, with a target-tracking scaling policy that launches an additional instance whenever average CPU utilization exceeds 70 percent, and a CloudWatch alarm (ec2-high-cpu-alarm) that visibly reflects the same threshold.

Objective

To let the web tier absorb sustained load automatically by adding capacity when CPU crosses 70 percent, rather than requiring manual intervention or risking degraded performance under load.

Prerequisites

- Task 9 completed (CPU monitoring already visible on the dashboard).
- An AMI or launch template capturing the configured public instance (Apache installed, security group attached).

Step-by-Step Implementation

Step 1: Create a Launch Template

EC2 Console -> Launch Templates -> Create launch template -> configure using lab-public-ec2's AMI, instance type, key pair, security group (lab-web-sg), and subnet settings:

Parameter	Value
Name	My-ASG
AMI	AMI created from public-EC2 (Apache pre-installed)
Instance type	t3.micro
Security group	default

Screenshot Evidence:

My-ASG (lt-0b5b261bf94a3bed9)
Actions ▾
Delete template

Launch template details

Launch template ID
lt-0b5b261bf94a3bed9

Launch template name
My-ASG

Default version
1

Owner
arn:aws:iam::565398309825:root

Details
Versions
Template tags

Launch template version details
Actions ▾
Delete template version

Version
1 (Default)

Description
-

Date created
2026-07-30T12:33:02.000Z

Created by
arn:aws:iam::565398309825:root

Instance details
Storage
Resource tags
Network interfaces
Advanced details

AMI ID
ami-02b64aa047cb5edf5

Instance type
t3.micro

Availability Zone
-

Availability Zone Id
-

Key pair name
multitool-key

Security groups
-

Security group IDs
sg-0e095614194496251

Step 2: Create the Auto Scaling Group

EC2 Console -> Auto Scaling Groups -> Create Auto Scaling group -> configure:

Parameter	Value
Name	Test-ASG
Launch template	My-ASG
VPC subnets	public-subnet
Attach to load balancer	lab-tg (existing target group)
Desired / Min / Max capacity	1 / 1 / 3

Screenshot Evidence:

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 1
Choose launch template
Step 2
Choose instance launch options
Step 3 - optional
Integrate with other services
Step 4 - optional
Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review

Choose launch template
Specify a launch template that contains settings common to all EC2 instances that are launched by this Auto Scaling group.

Name
Auto Scaling group name
Enter a name to identify the group.
Test-ASG
Must be unique to this account in the current Region and no more than 255 characters.

Launch template
For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.

Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.
My-ASG
Create a launch template

Version
1

Launch template

Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

My-ASG

Create a launch template

Version

Default (1)

Create a launch template version

Description

-

Launch template

My-ASG
lt-0b5b261bf94a3bed9

Instance type

t3.micro

AMI ID

ami-02b64aa047cb5edf5

Security groups

-

Request Spot Instances

No

Key pair name

multitool-key

Security group IDs

sg-0e095614194496251

Additional details

Storage (volumes)

-

Date created

Thu Jul 30 2026 18:03:02 GMT+0530 (India Standard Time)

Network

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC

Choose the VPC that defines the virtual network for your Auto Scaling group.

vpc-0ea200865b2aa88e0 (My-vpc)
192.168.0.0/20

Create a VPC

Availability Zones and subnets

Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

use1-az1 (us-east-1a) | subnet-049df76aaa0186157 (public-subnet)
192.168.0.0/24

Create a subnet

Availability Zone distribution - new

Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

Balanced best effort

If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

Balanced only

If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

Reservations Then Balanced

Prioritizes launching into Capacity Reservations, distributing across AZs with available reservations. When reservations are fully utilized, falls back to balanced on-demand.

Desired capacity

Specify your group size.

1

Scaling

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

2

Equal or less than desired capacity

Max desired capacity

5

Equal or greater than desired capacity

Automatic scaling - optional

Choose whether to use a target tracking policy

No scaling policies

Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

Target tracking scaling policy

Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Instance maintenance policy

Step 3: Add a Target-Tracking Scaling Policy

Test-ASG -> Automatic scaling -> Create dynamic scaling policy -> configure:

Parameter	Value
Policy type	Target tracking scaling
Metric type	Average CPU Utilization
Target value	70
Instance warmup	300 seconds

Screenshot Evidence:

Create dynamic scaling policy

Policy type
Target tracking scaling

Scaling policy name
Target Tracking Policy

Metric type [Info](#)
Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.
Average CPU utilization

Target value
70

Instance warmup [Info](#)
300 seconds

☐ Disable scale in to create only a scale-out policy

[Cancel](#) [Create](#)

Step 4: Create the Companion CPU Alarm

CloudWatch Console -> Alarms -> Create alarm -> Select metric -> EC2 -> CPUUtilization -> configure:

Parameter	Value
Alarm name	ec2-high-cpu-alarm
Statistic / Period	Average / 5 minutes
Condition	Greater than 70
Datapoints to alarm	2 out of 2

Screenshot Evidence:

Metric [Edit](#)

Graph
This alarm will trigger when the blue line goes above the red line for 2 datapoints within 10 minutes.

Percent
70% — 70
39%
8.01%

10:00 10:30 11:00 11:30 12:00 12:30

— CPUUtilization

Namespace
AWS/EC2

Metric name
CPUUtilization

AutoScalingGroupName
Test-ASG

Statistic
Average

Period
5 minutes

Conditions

Threshold type

☒ Static
Use a value as a threshold

☐ Anomaly detection
Use a band as a threshold

Whenever CPUUtilization is...

Define the alarm condition.

☒ Greater
> threshold

☐ Greater/Equal
≥ threshold

☐ Lower/Equal
≤ threshold

☐ Lower
< threshold

than...

Define the threshold value.

70

Must be a number.

▼ Additional configuration

Datapoints to alarm

Define the number of datapoints within the evaluation period that must be breaching to cause the alarm to go to ALARM state.

2 out of 2

Missing data treatment

ec2-high-cpu-alarm

Actions

View

Explore related

Investigate

Details

Tags

Mute rules - new

Actions

History

Parent alarms

Details

Name ec2-high-cpu-alarm	State Insufficient data	Namespace AWS/EC2	Datapoints to alarm 2 out of 2
Type Metric alarm	Threshold CPUUtilization > 70 for 2 datapoints within 10 minutes	Metric name CPUUtilization	Missing data treatment Treat missing data as missing
Description No description	Actions Actions enabled	AutoScalingGroupName Test-ASG	Percentiles with low samples evaluate
	Last state update 2026-07-30 12:48:00 (UTC)	Statistic Average	ARN arn:aws:cloudwatch:us-east-1:565398309825:alarm:ec2-high-cpu-alarm
► View EventBridge rule			

Validation Steps

- The Auto Scaling Group shows one healthy instance registered in lab-tg under normal load.
- Generating sustained CPU load above 70 percent (for example with a stress-test tool) causes the Activity tab to show a scale-out event and a second instance registers itself in lab-tg within a few minutes.
- ec2-high-cpu-alarm transitions from OK to In alarm at the same time the scaling activity occurs.

Explanation

A target-tracking policy continuously compares the group's average CPUUtilization against the 70 percent target and adjusts desired capacity up or down to keep the metric near that target, launching new instances from the same launch template and automatically registering them with the attached target group. The companion CloudWatch

alarm evaluates the identical metric and threshold, giving an independent, human-visible signal of the same condition that is driving the scaling action.

Real-Time Use Case

This is the standard defense against traffic spikes for a stateless web tier: instead of over-provisioning for peak load at all times, the group runs lean under normal conditions and adds capacity automatically the moment sustained demand pushes CPU past a safe threshold, then scales back in once load subsides.

Common Errors and Troubleshooting

Error	Cause	Resolution
New instance never registers as healthy in the target group	Launch template's security group or subnet does not allow ALB health checks	Confirm the launch template uses lab-web-sg and a subnet reachable by the ALB
Scaling never triggers under real load	Target value set too high, or instance warmup too long relative to the load spike	Lower the target value or verify load is sustained long enough to cross the warmup period
Group scales out repeatedly then immediately back in	Load is spiky rather than sustained, causing policy oscillation	Increase the instance warmup time or evaluate a step scaling policy with a cooldown period instead
